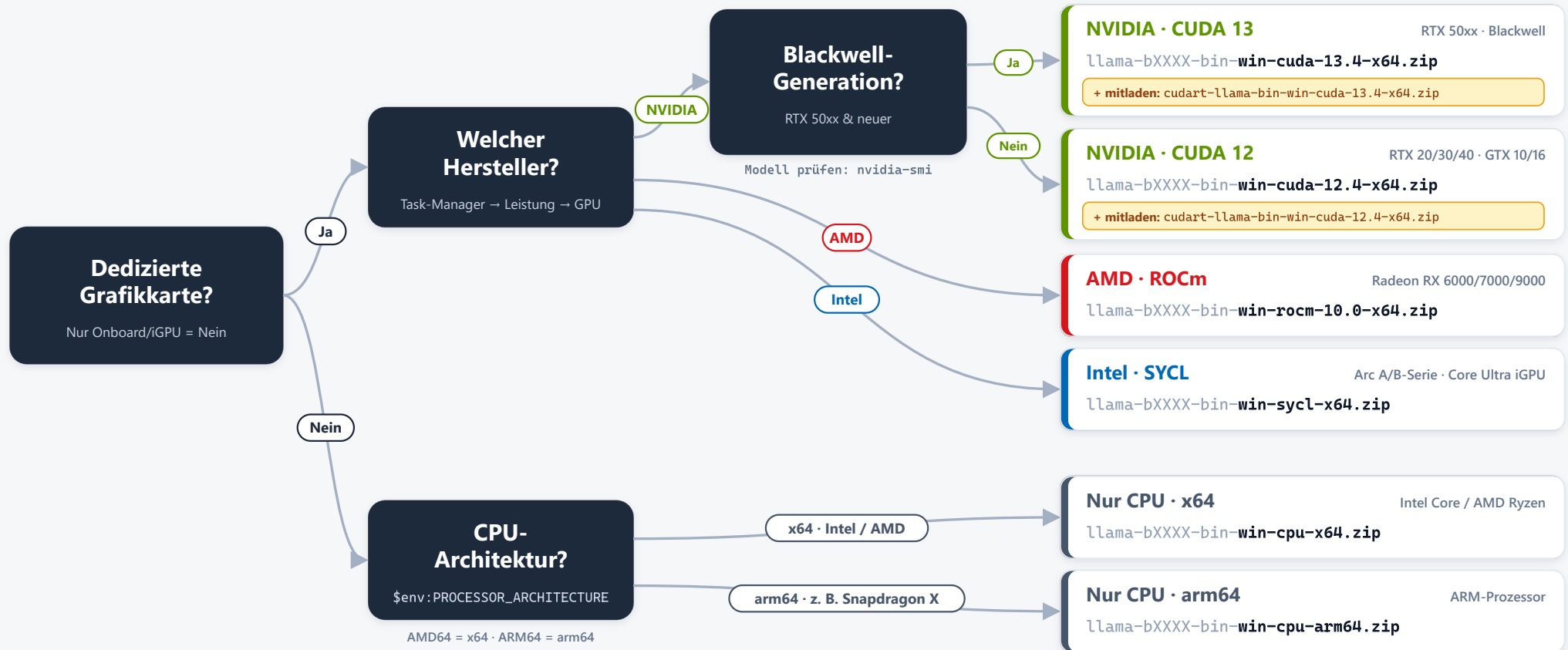


llama.cpp Download-Kompass

WINDOWS

Quelle: github.com/ggml-org/llama.cpp/releases → neuestes Release → „Assets“ aufklappen

Alle Dateien heißen `llama-bXXXX-bin-...` (XXXX = aktuelle Build-Nr., z. B. b11377) ■ Gelb = zweites Archiv (CUDA-Runtime) laden & in DENSELBEIN Ordner entpacken



1

Herunterladen

Datei(en) aus dem Baum oben laden

2

Entpacken – alles in EINEN Ordner

.zip entpacken (Rechtsklick → Alle extrahieren)

3

Modell (.gguf) dazu → Terminal → starten

`./llama-server.exe -m modell.gguf` → localhost:8080

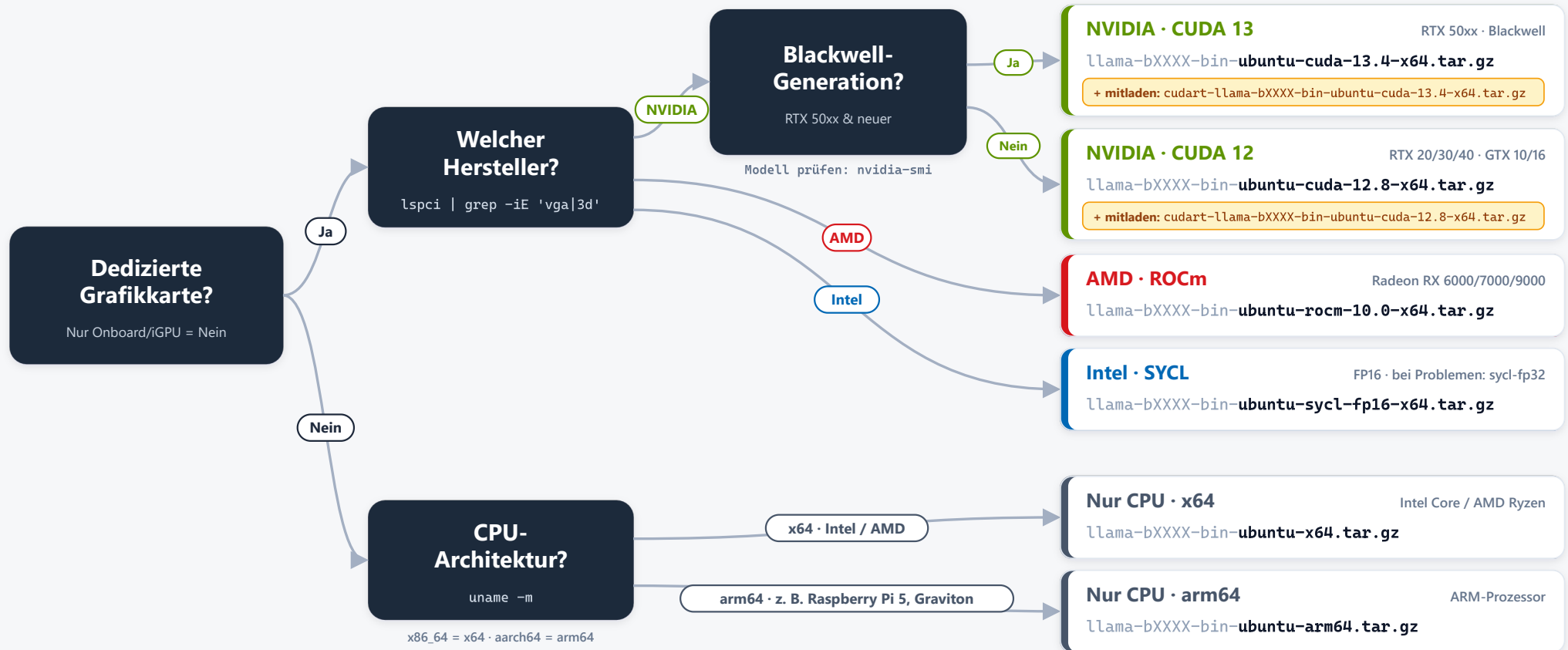
Bewusst weggelassen: Vulkan (Universal-Fallback, z. B. ältere AMD-Karten / iGPU), OpenVINO, OpenCL-Adreno, NVIDIA auf arm64 (win-cuda-13.4-arm64), ROCm/SYCL: aktuellen GPU-Treiber installieren.

llama.cpp Download-Kompass

LINUX · UBUNTU

Quelle: github.com/ggml-org/llama.cpp/releases → neuestes Release → „Assets“ aufklappen

Alle Dateien heißen `llama-bXXXX-bin-...` (XXXX = aktuelle Build-Nr., z. B. b11377) ■ Gelb = zweites Archiv (CUDA-Runtime) laden & in DENSELBEN Ordner entpacken



1

Herunterladen

Datei(en) aus dem Baum oben laden

2

Entpacken – alles in EINEN Ordner

je Archiv: `tar -xzf <datei>.tar.gz` (cudart ebenso)

3

Modell (.gguf) dazu → Terminal → starten

`./llama-server -m modell.gguf` → localhost:8080

Bewusst weggelassen: Vulkan (Universal-Fallback), OpenVINO, s390x, Snapdragon. NVIDIA auf arm64 (DGX Spark, Jetson Thor): `ubuntu-cuda-13.4-arm64` + passende cudart. ROCm/SYCL: Treiber bzw. Runtime installiert haben.

llama.cpp Download-Kompass

Quelle: github.com/ggml-org/llama.cpp/releases → neuestes Release → „Assets“ aufklappen

Desktop = Terminal-Weg mit Release-Dateien · Smartphone = **fertige Wrapper-App nehmen** (laufen unter der Haube auf llama.cpp / GGUF)

APPLE & MOBILE

macOS

Terminal-Weg

Welcher Chip?

→ Über diesen Mac

M1 – M5

Apple Silicon

empfohlen

llama-bXXXX-bin-macos-arm64.tar.gz
Metal-GPU automatisch aktiv · RAM = VRAM

Intel

Intel-Mac

primär CPU

llama-bXXXX-bin-macos-x64.tar.gz
eher kleine Modelle (≤ 8B, Q4)

„Kann nicht geöffnet werden“? Gatekeeper-Sperre im Ordner lösen:

```
xattr -dr com.apple.quarantine .
```

1

Bequemer: Homebrew

```
brew install llama.cpp
```

2

Starten

```
./llama-server -m modell.gguf
```

Gibt es auch: macos-arm64 „KleidiAI“ – aktuell deaktiviert, ignorieren.

iOS / iPadOS

App-Weg

EMPFEHLUNG · Wrapper-App

Pocket AI Lab

App Store

im App Store suchen

- Kein Terminal, keine Binaries
- App lädt & verwaltet Modelle
- Läuft komplett offline
- Realistisch: 1–4B in Q4
- Geräte mit 8 GB+ RAM ideal

NUR FÜR ENTWICKLER / BASTLER

Eigene App bauen: ...-xcframework.zip
→ in Xcode einbinden

Android

App-Weg

EMPFEHLUNG · Wrapper-App

LM Playground

lmplayground.app · kostenlos · Open Source

- Kein Terminal, keine Binaries
- App lädt & verwaltet Modelle
- Läuft komplett offline
- Realistisch: 1–4B in Q4
- Geräte mit 8 GB+ RAM ideal

NUR FÜR ENTWICKLER / BASTLER

Termux / adb: android-arm64
Snapdragon: ...-arm64-snapdragon
(Adreno GPU + Hexagon NPU)

Warum auf dem Smartphone eine Wrapper-App?

iOS/Android haben kein offenes Terminal & sandboxen Apps · App übernimmt Download, Speicher & Thermik · gleiche GGUF-Modelle wie am PC